Politechnika Krakowska
im. Tadeusza Kościuszki

Wstęp do Cyberbezpieczeństwa

Laboratorium 3

Nr albumu: D/161212

Jakub Kozieński

Grupa: CY2

Implementacja algorytmu RSA

1.1. Wczytanie wiadomości z pliku
Poniższy kod otwiera plik tekstowy i zwraca jego zawartość jako ciąg znaków. Alfabet ograniczony jest do 26 liter (a-z) - wszystkie inne znaki (spacje, cyfry, znaki specjalne) są ignorowane na etapie przetwarzania.

```
# --- Wczytanie wiadomości ---
input_file = "message.txt"
with open(input_file, "r", encoding="utf-8") as f:
    message = f.read()
print(f"\nWczytano wiadomość z pliku '{input_file}'")
print(f"Oryginalna wiadomość: '{message}'")
```

1.2. Podział tekstu na bloki 10 znakowe
Wczytuje tekst, czyszczę do liter a-z, dzielę na bloki po *block_size* znaków.
Ostatni blok jest uzupełniany literą 'a' (wartość 0) jeśli jest krótszy.

```
def read_and_prepare(text, block_size=10):
    clean = ''.join(c for c in text.lower() if 'a' <= c <= 'z')
    if not clean:
        raise ValueError("Tekst nie zawiera liter a-z!")

    blocks = []
    lengths = []
    for i in range(0, len(clean), block_size):
        block = clean[i:i + block_size]
        lengths.append(len(block))
        # Uzupełnienie ostatniego bloku
        block = block.ljust(block_size, 'a')
        blocks.append(block)
    return blocks, lengths
```

1.3. Zamiana bloków na liczby

Każdy blok znaków jest zamieniany na liczbę całkowitą w systemie o podstawie 26. Litera „a" odpowiada 0, „b" - 1, ..., „z" - 25. Operacja jest odwracalna, co pozwala na odtworzenie tekstu po odszyfrowaniu.

```python
def char_to_num(c):

    return ord(c.lower()) - ord('a')

def num_to_char(n):
    return chr(n + ord('a'))

def block_to_number(block):
    result = 0
    for i, c in enumerate(block):
        result += char_to_num(c) * (ALPHABET_SIZE ** i)
    return result

def number_to_block(number, length):
    chars = []
    for _ in range(length):
        chars.append(num_to_char(number % ALPHABET_SIZE))
        number //= ALPHABET_SIZE
    return ''.join(chars)
```

1.4. Wygenerowanie klucza o długości 768 bitów

Generowanie klucza zostało zrealizowane zgodnie z algorytmem RSA: wybrano dwie różne liczby pierwsze p i q o długości 384 bitów każda (łącznie n = p*q ma 768 bitów), obliczono phi = (p-1)*(q-1), dobrano wykładnik publiczny e, a następnie obliczono wykładnik prywatny d = e^(-1) mod phi za pomocą algorytmu Euklidesa.

Uzasadnij, dlaczego wybrałeś właśnie taką wartość e (np. 3, 17, 65537)

Jako wykładnik publiczny wybrano e = 65537 (2^16 + 1), ponieważ jest to liczba pierwsza, w zapisie binarnym ma tylko dwa bity równe 1 (1000000000000001), dzięki czemu algorytm szybkiego potęgowania modularnego wymaga zaledwie 16 mnożeń, co przyspieszy szyfrowanie oraz dla bardzo małych e (np. e = 3) możliwe są ataki matematyczne polegające na tym, że ta sama wiadomość zaszyfrowana wieloma różnymi kluczami publicznymi pozwala na odtworzenie plaintextu bez znajomości klucza prywatnego.

```python
def generate_rsa_keys(key_bits=KEY_BITS):
    p_bits = key_bits // 2 - 64
    q_bits = key_bits // 2 + 64

    # Krok 1: dwie różne liczby pierwsze
    while True:
        p = generate_prime(p_bits)
        q = generate_prime(q_bits)
        if p != q:
            break

    # Krok 2
    n = p * q

    # Krok 3
    phi = (p - 1) * (q - 1)

    # Krok 4
    e = 65537
    if e >= phi or math.gcd(e, phi) != 1:
        raise ValueError(f"Wybrane e={e} nie spełnia warunków RSA dla tego klucza.")

    # Krok 5
    d = pow(e, -1, phi)

    return n, e, d, p, q
```

1.5. Zaszyfrowanie wiadomości blok po bloku

Każdy blok tekstowy zamieniany jest na liczbę m, a następnie szyfrowany wzorem c = m^e mod n. Warunkiem poprawności jest spełnienie nierówności m < n - jeśli nie jest spełniona, program zgłasza błąd. Do obliczeń używana jest funkcja pow(m, e, n), realizująca szybkie potęgowanie modularne.

```python
def encrypt_block(m, e, n):
    return pow(m, e, n)


def encrypt_message(blocks, e, n, block_size=10):
    encrypted = []
    for block in blocks:
        m = block_to_number(block)
        if m >= n:
            raise ValueError(
                f"Błąd: wartość bloku m={m} >= n={n}. "
                "Zwiększ długość klucza lub zmniejsz rozmiar bloku."
            )
        c = encrypt_block(m, e, n)
        encrypted.append(c)
    return encrypted
```

1.6. Odszyfrowanie wiadomości blok po bloku

Deszyfrowanie przebiega symetrycznie do szyfrowania: każdy szyfrogram c jest przetwarzany wzorem m = c^d mod n. Wynikowa liczba m zamieniana jest z powrotem na blok znaków przy użyciu operacji odwrotnej do kodowania pozycyjnego.

```python
def decrypt_block(c, d, n):
    return pow(c, d, n)

def decrypt_message(encrypted, d, n, lengths, block_size=10):
    decrypted_blocks = []
    for c, length in zip(encrypted, lengths):
        m = decrypt_block(c, d, n)
        block = number_to_block(m, block_size)

        # Przycinanie do oryginalnej długości bloku
        decrypted_blocks.append(block[:length])
    return ''.join(decrypted_blocks)
```

1.7. Sprawdzenie zgodności wiadomości po odszyfrowaniu

Funkcja *verify()* porównuje każdy oryginalny blok tekstu z blokiem odzyskanym po odszyfrowaniu. Wynik każdego porównania wypisywany jest jako [OK] lub [BŁĄD].

```python
def verify(original_blocks, lengths, encrypted, d, n, block_size=10):
    all_ok = True
    print("\n  Weryfikacja blok po bloku  ")
    for i, (block, c, length) in enumerate(zip(original_blocks, encrypted, lengths)):
        m_dec = decrypt_block(c, d, n)
        block_dec = number_to_block(m_dec, block_size)[:length]
        original = block[:length]
        ok = (original == block_dec)
        if not ok:
            all_ok = False
        status = "OK" if ok else "BŁĄD"
        print(f"  Blok {i+1:2d}: '{original}' -> szyfrogram={c} -> '{block_dec}' [{status}]")
    return all_ok
```

Dlaczego RSA wymaga, aby m < n (na podstawie Twojego przykładu)?

```
  Demo: porównanie m < n i m >= n
  [m < n]  m = 42, c = m^e mod n = 8105971372624837517396261790739483405204150459068221982117968668542797534082037299821212414778392457279150602692195840107449054359040705571285521823018020130832260165
721904188852036703225613822398353425982781479 27, m' = 42, m == m': True
  [m >= n] m = n+42, c = m^e mod n = 8105971372624837517396261790739483405204150459068221982117968668542797534082037299821212414778392457279150602692195840107449054359040705571285521823018020130832260
3672190418885203670322561382239835342598278147927, m' = 42, m == m': False
  Oba szyfrogramy identyczne: True  ← ta sama liczba, nie do odróżnienia!
```

W moim przykładzie zaszyfrowano dwie różne liczby: m = 42 oraz m = n + 42. Obie dały identyczny szyfrogram c. To przez to, że liczby 42 i n + 42 różnią się o dokładnie n, więc ich reszta z dzielenia przez n jest taka sama i wynosi 42. RSA traktuje obie wiadomości jako tę samą liczbę. I w wyniku odszyfrowanie zwróciło 42 w obu przypadkach - oryginalna wiadomość n + 42 została bezpowrotnie utracona.

Dlatego RSA wymaga m < n - wtedy każda wiadomość ma unikalny szyfrogram i odszyfrowanie zawsze odtwarza oryginał.

Dlaczego szyfrowanie identycznych bloków daje takie same szyfrogramy?
RSA szyfruje wzorem gdzie parametry są stałe, więc dla identycznego bloku wynik jest zawsze taki sam. Ten algorytm jest w pełni deterministyczny i nie zawiera żadnego elementu losowego. Przez to dwa identyczne bloki tekstu zawsze dają identyczny szyfrogram.

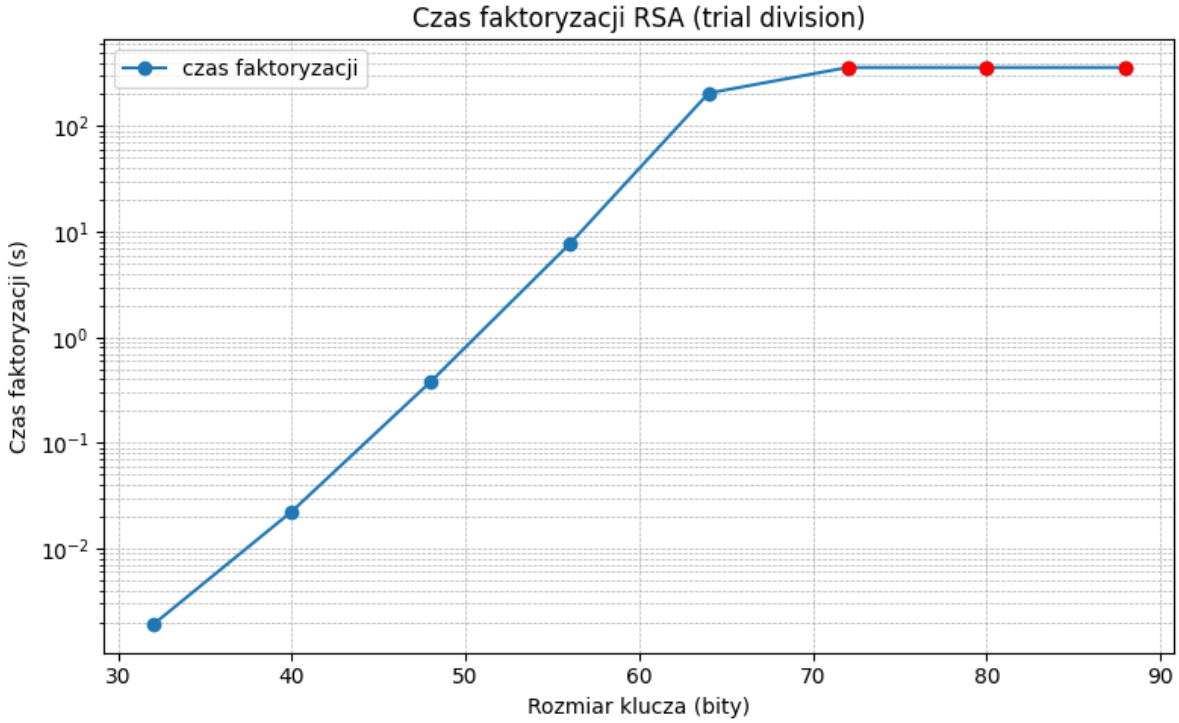
Co to jest padding (np. OAEP) i jak rozwiązuje ten problem?
To technika wstawienia dodatkowych danych przed szyfrowaniem wiadomości. Jego głównym celem jest wprowadzenie losowości, która sprawia że szyfrowanie przestaje być deterministyczne.
Np. OAEP przed szyfrowaniem miesza wiadomość z losowym seedem za pomocą funkcji MGF1, przez co to samo m za każdym razem daje inny szyfrogram.

Faktoryzacja RSA

1. Zanotuj dla każdego: rozmiar klucza (bity), metoda faktoryzacji, czas łamania (s/ms), liczba iteracji, czy udało się znaleźć p i q?

```
================================================================
FAKTORYZACJA RSA - Test czasowy (trial division, limit czasu)
================================================================
  Bity |     Czas [s] |       Iteracje |  Sukces
----------------------------------------------------------------
    32 |     0.001893 |         19,509 |     TAK
    40 |     0.022445 |        268,385 |     TAK
    48 |     0.380722 |      4,548,734 |     TAK
    56 |     7.674932 |     87,852,906 |     TAK
    64 |   205.942963 |  1,729,517,844 |     TAK
    72 |   360.002022 |  2,811,500,000 |     NIE (TIMEOUT)
    80 |   360.003981 |  2,771,500,000 |     NIE (TIMEOUT)
    88 |   360.032532 |  2,774,500,000 |     NIE (TIMEOUT)
```

2. Narysuj wykres: rozmiar klucza (bity) → czas faktoryzacji
   Wykres przedstawia zależność czasu faktoryzacji od rozmiaru klucza w skali logarytmicznej. Widać wzrost każde kolejne 8 bitów wydłuża czas około 10-krotnie. Punkty czerwone oznaczają przekroczenie limitu czasu (360 s).



3. Spróbuj dopasować co najmniej dwie różne krzywe (np. liniową, potęgową, wykładniczą, logarytmiczną)

```
Dopasowanie krzywych
Potęgowa:    t = 1.0177e-28 * x^16.585   R2 = 0.9763
Wykładnicza: t = 1.3100e-08 * e^(0.3629*x)  R2 = 0.9972
```

4. Wybierz tę, która najlepiej pasuje do Twoich danych (np. najmniejszy błąd / najwyższy R^2)

   Różnica między dopasowaniem obu krzywych jest minimalna, jednak funkcja wykładnicza dopasowuje się minimalnie lepiej. Tak mała różnica może wynikać z małej ilości danych, bo dysponujemy tylko 5 punktami (32–64 bity), ponieważ pozostałe przekroczyły limit czasu.

5. Uzasadnij, dlaczego wybrałeś akurat ten typ funkcji, a nie inny.

   Wybrałem funkcję wykładniczą, ponieważ algorytm trial division musi sprawdzać dzielniki aż do $\sqrt{n}$. Gdy klucz rośnie o 1 bit, n się podwaja, a więc $\sqrt{n}$ rośnie o $\sqrt{2} \approx 1.414$, czyli o około 41%. Każdy dodatkowy bit mnoży ilość pracy przez stałą wartość a to jest wzrost wykładniczy.

6. Spróbuj ekstrapolować i oszacować orientacyjny czas łamania klucza: 96- bitowego, 128-bitowego, 256-bitowego

   Złożoność metody trial division wynosi $O(\sqrt{n})$. Stosunek czasów jest równy stosunkowi złożoności, co pozwala wyprowadzić wzór $t2 = t1 \cdot 2^{(\Delta k/2)}$, gdzie t1 to znany czas, a Δk to różnica w liczbie bitów. Czas faktoryzacji klucza 64-bitowego wyniósł t = 205.942963 s, prognozowane czasy dla większych kluczy wynoszą:

   72 bity: $\sim 3.295 \cdot 10^{3}$ s (~55 minut)

   80 bitów: $\sim 5.272 \cdot 10^{4}$ s (~15 godzin)

   88 bitów: $\sim 8.435 \cdot 10^{5}$ s (~10 dni)

   96 bitów: $\sim 1.350 \cdot 10^{7}$ s (~5 miesięcy)

   128 bitów: $\sim 8.845 \cdot 10^{11}$ s (~28 029 lat)

   256 bitów: $\sim 1.632 \cdot 10^{31}$ s ($\sim 5.17 \cdot 10^{23}$ lat)

7. Czy można ufać tym prognozom przy dużych kluczach (np. 1024, 2048 bitów)?

   Mój model oparty jest na algorytmie trial division, który jest metodą naiwną i nikt w praktyce nie używa go do łamania kluczy RSA. Rzeczywiste ataki kryptograficzne stosują znacznie efektywniejsze metody, które mają nawet sub-wykładniczą złożoność. Dodatkowo, ekstrapolacja wzoru $t2 = t1 \cdot 2^{(\Delta k/2)}$ zbudowanego na danych z zakresu 32-64 bitów jest rozciągana o setki bitów, co kumuluje błąd. Prognozy te mają jedynie charakter poglądowy i nie odzwierciedlają rzeczywistej trudności łamania kluczy dużych kluczy.